

# 01

## PROTOCOL

---

### Arcium Integration Note

How Protocol 01 uses Arcium MPC, where it sits in the privacy stack, and how the on-chain flow stays inside Solana's transaction limits.

**9**

MPC  
CIRCUITS

**3**

WIRED IN  
APP

**12**

ACCOUNTS /  
QUEUE

**200B**

MAX PAYLOAD

PREPARED FOR THE ARCIUM TEAM • 2026-05-12

## 01 The gap that ZK doesn't fill

Protocol 01 is the privacy layer Solana doesn't have yet. The core of our stack is post-quantum ZK-STARKs, stealth addresses, and on-chain verification. That covers the cases where a single user proves a single statement about their own state. It does not cover what Arcium does.

### ZK-STARKs

Winterfell, Poseidon over Goldilocks. Six AIRs verified on-chain in under 1.4M CU. Shield, unshield, transfer, balance, pool, Merkle update.

Solo prover

### Stealth addresses

X25519 plus ML-KEM-768 (NIST PQC). One-time addresses, sender-receiver unlinkable, post-quantum receive side.

Receive privacy

### Arcium MPC

Joint computation over N parties. Sealed inputs. Threshold reveal. Everything ZK alone cannot express cleanly.

Multi-party

We did not want to fake MPC with a trusted off-chain service, and we did not want to skip the multi-party use cases entirely. Arcium gave us a real path: secret shares over an actual network, threshold reveal, signed callbacks on Solana.

### Where each primitive owns the problem

| PROBLEM                     | BEST PRIMITIVE | WHY                                                                                  |
|-----------------------------|----------------|--------------------------------------------------------------------------------------|
| User proves their own state | ZK-STARK       | Single prover, single statement, single tx, verifier on-chain.                       |
| Recipient identity privacy  | Stealth        | Sender cannot guess the recipient's actual address. ECDH derivation.                 |
| Inputs from N parties       | Arcium MPC     | No single party holds all inputs, so no single prover can produce a ZK proof.        |
| Sealed inputs after reveal  | Arcium MPC     | Only the aggregated result becomes public. Individual inputs stay encrypted forever. |
| Threshold operations        | Arcium MPC     | N-of-M reveal. No single node ever recovers the plaintext.                           |

## 02 Nine confidential computations, eight SDK modules

One Solana program ( `p01_arciium` , ~3.5k lines including encrypted instructions). One TypeScript SDK ( `@protocol-01/arciium-sdk` , ~2400 lines, organised by use case rather than raw circuit). Every circuit has a typed queue plus callback pair, registered through `init_computation_definition_accounts` .

| MODULE     | ARCIUM CIRCUIT                                                                  | WHAT IT DOES                                                                                                                                       | STATUS    |
|------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Relay      | <code>threshold_decrypt</code>                                                  | N-of-M MPC nodes jointly decrypt and submit a Solana transaction. The user wallet never appears as the fee payer or as a signer on-chain.          | Wired     |
| Nullifier  | <code>nullifier_commit</code>                                                   | SHA3 commitment over a nullifier computed inside MPC. Prevents double-spend without revealing the actual nullifier value.                          | Wired     |
| Registry   | <code>private_lookup</code>                                                     | Look up a meta-address without revealing which key is being queried, so RPC nodes cannot correlate the lookup.                                     | Wired     |
| Stealth    | <code>register_viewing_key</code><br><code>stealth_scan_single</code>           | Viewing keys stored under MPC. Scanning runs so that no single node ever reconstructs the viewing key in clear.                                    | SDK ready |
| Audit      | <code>balance_audit</code><br><code>finalize_audit</code>                       | Confidential aggregation of many encrypted balances, then threshold reveal of just the total. Solvency proof without exposing individual balances. | SDK ready |
| Governance | <code>private_vote</code><br><code>finalize_tally</code><br>(+ binary variants) | Encrypted ballots, tally computed inside MPC, only the totals released. Binary variant optimised for yes-no votes.                                 | SDK ready |
| Auction    | <code>sealed_bid_auction</code><br><code>finalize_auction</code>                | Encrypted bid matching. Winner revealed by MPC. Losing bids stay confidential after settlement.                                                    | SDK ready |

### Wired today

Three modules live behind an opt-in flag in the mobile app ( `apps/mobile/services/arciium/` plus `arciiumStore.mpcEnabled` ). They meaningfully change the trust model of the wallet.

### SDK ready

Six circuit families ship as ready-to-call SDK modules but do not yet have a deployed UI caller. They are on the roadmap, not vaporware.

## 03 Three phases, two transactions, one async pattern

Every Arcium interaction follows the same shape. This is also the answer to the transaction-size question: nothing tries to fit submission, computation, and result into a single Solana tx.

### 1 Submission (user-signed Solana tx)

Client calls `ArciumClient.initialize()` to establish an x25519 shared secret with the MXE. It encrypts inputs with RescueCipher, picks a fresh 64-bit `computation_offset`, and invokes the matching queue instruction. The program CPIs into the Arcium runtime via `queue_computation(...)`.

### 2 Off-chain MPC (no Solana tx)

ARX nodes pick up the queued computation and run the circuit on secret shares. Cerberus guarantees that as long as at least one node is honest, no single node ever sees the plaintext inputs. Typically completes in a few seconds.

### 3 Callback (Arcium-signed Solana tx)

The cluster posts a callback transaction with a `SignedComputationOutputs<T>` payload. Our `*_callback` handler verifies the signature against `cluster_account` and `computation_account`, then emits a typed event or updates a PDA. The user decrypts revealed values locally.

## Transaction-size analysis

Solana enforces 1232 bytes per serialised message and 64 unique accounts per tx (35 writable in v0). The Arcium queue instructions sit far below both limits.

### Accounts in a queue ix

1. payer (signer, mut)
2. sign\_pda\_account
3. mxe\_account
4. mempool\_account (mut)
5. executing\_pool (mut)
6. computation\_account (mut)
7. comp\_def\_account
8. cluster\_account (mut)
9. pool\_account (mut, fixed)
10. clock\_account (mut, fixed)
11. system\_program
12. arcium\_program

**12 / 64** far below the ceiling

### Data payload per submission

|                           |             |
|---------------------------|-------------|
| computation_offset        | 8 B         |
| ephemeral x25519 pubkey   | 32 B        |
| nonce (u128)              | 16 B        |
| encrypted blocks (1 to 5) | 32 to 160 B |

**Total ix data** **64 to 216 B**

**~600 B / 1232 B** full serialised tx, worst case

**Why this works.** The submission and the callback are two separate transactions. The off-chain MPC happens between them. We never try to fit the whole computation into one tx, which is why the 1232-byte and 64-account limits stay theoretical concerns in our case. The largest observed real submission, with five encrypted arguments, lands around 400 bytes serialised.

## 04 Honest status

We try not to overclaim. The wiring is real. The full per-circuit devnet validation pass is still in progress.

### What works today

- The Rust program compiles. The encrypted instructions in `encrypted-ixs/src/lib.rs` build through the Arcium toolchain.
- The TypeScript SDK is complete, builds green, exports all nine circuit families with typed wrappers.
- The mobile app wires three modules (`threshold_decrypt`, `nullifier_commit`, `private_lookup`) into user-facing flows behind an opt-in flag.
- The previously deployed program ID on devnet, `9kMjmVMYxBa8V9D1aoEjZtUNXTe2gjfzYdKLycn7JvgQ`, remains live and resolvable.

### What is still in progress

- A per-circuit devnet validation pass with recorded transaction signatures for each callback is pending. The wiring exists; the public artifacts do not yet.
- `p01_arcium` is temporarily excluded from `[programs.devnet]` and `[programs.localnet]` in `Anchor.toml` (commits `70fbab9` and `2bc6b0f`, 2026-05-12). The devnet deployment is frozen at the program ID above and continues to run.

### Why `p01_arcium` is currently out of the CI build

Three reasons stack. None of them are about the protocol design, they are toolchain integration items.

#### **.arcis artifacts**

Compiled circuit binaries (`.arcis`) are produced by the Arcium toolchain at build time and are not checked into git. CI does not yet run the Arcis compiler.

#### **Anchor program tables**

`anchor build` walks the `[programs.*]` map independently of the Cargo workspace exclude rules, so simply removing the crate from the workspace did not unblock the whole-workspace build.

#### **SBF stack overflow**

`arcium-client` 0.9.7 has a stack frame that overflows the 4 KB SBF stack limit, which blocked the SBF build of every other program in the workspace.

Resolution path: bring the Arcis compiler into our CI image, then re-enable `p01_arcium` in the Anchor program tables and run the full per-circuit validation pass. This is the next milestone on the Arcium side.

# 05 What we would like to learn from you

Protocol 01 is solo built (Volta Team, end of January 2026 to today). We picked Arcium deliberately, not as a checkbox. Six of the nine circuits are SDK-ready scaffolding for now because they belong on the roadmap, not to inflate a feature count. The three wired today are the ones that meaningfully change the trust model of the mobile app.

**Where feedback would help most**

- Arcis idioms for circuits we have not yet pushed to devnet, especially `stealth_scan_single` and the audit pair.
- Best practice for integrating the Arcis compiler into a multi-program Anchor workspace CI.
- Whether our 12-account queue layout matches what you would recommend, or if we are missing an idiomatic pattern.

**What we are happy to share**

- Full source for `p01_arcium` and the SDK modules.
- Concrete numbers from devnet (CU usage, queue tx size, callback latency) once the validation pass is complete.
- Any pattern that turned out useful when wrapping Arcium for a multi-platform client.

## Where to look in the repo

|                                                 |                                                                                                                    |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>programs/p01_arcium/src/lib.rs</code>     | Program entrypoints, queue and callback pairs (lines 100 to 1430).                                                 |
| <code>programs/p01_arcium/encrypted-ixs/</code> | The actual circuits in Arcis (687 lines).                                                                          |
| <code>packages/arcium-sdk/src/client.ts</code>  | Encryption pipeline, ephemeral key rotation, PDA derivation.                                                       |
| <code>packages/arcium-sdk/src/{module}/</code>  | Eight typed wrappers (relay, nullifier, registry, audit, governance, auction, stealth, plus one not covered here). |
| <code>apps/mobile/services/arcium/</code>       | Live wiring for the three opt-in flows.                                                                            |

## Thank you for reading

If anything in this note reads wrong from the inside, we would rather hear it now than ship it. Feedback, integration hints, or a pointer toward the right Arcis idioms are very welcome.

protocol-01.dev · [github.com/IsSlashy/Protocol-01](https://github.com/IsSlashy/Protocol-01)